

Clase 10 — Unidad II

Patrones de diseño: Singleton, Factory y Repository

Docentes: Franco Navarro y Agustín Almirón.

Este documento es la guía teórica de la clase. Los ejemplos de código están en el cuaderno de Colab que acompaña al material.

Objetivos de la clase

- Comprender qué son los patrones de diseño y por qué nos resultan útiles.
- Identificar cuándo aplicar el patrón Singleton y reconocer sus riesgos.
- Aplicar el patrón Factory para desacoplar la creación de objetos.
- Entender el patrón Repository y su importancia en proyectos de datos.
- Resolver una actividad práctica integradora usando los tres patrones.

Temario

1. Repaso: ¿qué es un patrón de diseño?
2. Singleton: la única instancia.
3. Factory: fábricas de objetos.
4. Repository: abstracción del acceso a datos.
5. Comparación: cuándo usar cada uno.
6. Actividad práctica.
7. Material complementario.

1. Repaso: ¿qué es un patrón de diseño?

Un patrón de diseño es una solución probada y reutilizable a un problema recurrente en el diseño de software. No se trata de código listo para copiar y pegar, sino más bien de un plano que cada equipo adapta a su contexto particular.

Los patrones cumplen además una función de comunicación dentro del equipo. Cuando alguien dice «acá conviene un Singleton» o «usemos un Repository», quien escucha ya tiene una idea aproximada de la estructura que se va a implementar, sin necesidad de explicar todos los detalles. Funcionan como un vocabulario compartido entre desarrolladores.

Idea clave — los patrones son herramientas. Su valor depende de usarlos en el contexto correcto. Aplicar un patrón sin necesidad real (lo que se conoce como «patternitis») suele complicar el código en lugar de simplificarlo.

Clasificación clásica

La forma más habitual de clasificar los patrones agrupa según su propósito.

- **Patrones creacionales:** responden a la pregunta «¿cómo creo este objeto?». Ejemplos: Singleton, Factory, Builder, Prototype.
- **Patrones estructurales:** responden a «¿cómo combino estos objetos?». Ejemplos: Adapter, Decorator, Facade, Proxy.
- **Patrones de comportamiento:** responden a «¿cómo se comunican entre sí?». Ejemplos: Observer, Strategy, Command, Iterator.

En esta clase nos enfocamos en dos patrones creacionales del catálogo clásico (Singleton y Factory) y en un patrón arquitectural muy usado en sistemas modernos: Repository.

2. Singleton: la única instancia

Definición

El patrón Singleton garantiza que una clase tenga una única instancia a lo largo de toda la ejecución del programa, y provee un punto de acceso global a ella. Cada vez que se intenta crear un nuevo objeto de esa clase, en realidad se devuelve la misma instancia que se creó la primera vez.

¿Qué problema resuelve?

Hay recursos que, por su naturaleza, no tiene sentido duplicar dentro de una aplicación. Si distintos módulos creyeran tener su propio recurso, terminaríamos con estados inconsistentes, múltiples conexiones abiertas innecesariamente o caches que no comparten datos. Algunos casos típicos donde aparece esta necesidad:

- Un objeto de configuración global que centraliza parámetros como URL de la base de datos, modo debug o credenciales de API.
- Un logger que escribe en un único archivo y debe mantener el orden de las entradas.
- Una conexión a base de datos o un pool de conexiones que es costoso abrir y cerrar.
- Una caché compartida entre módulos del sistema.
- El acceso a un recurso de hardware único, como una impresora o un sensor.

Cómo funciona en la práctica

La idea central es interceptar el proceso de creación del objeto. Cuando alguien pide una instancia, la clase primero verifica si ya tiene una guardada. Si la tiene, la devuelve. Si no, crea una nueva, la guarda y la devuelve. A partir de ese momento, cualquier otra solicitud de creación devolverá la misma instancia.

En Python esto se puede implementar de varias formas: sobrescribiendo el método especial que controla la creación de objetos, usando un decorador, o aplicando una metaclasa. Todas estas variantes están desarrolladas con ejemplos ejecutables en el cuaderno de Colab que acompaña a esta clase.

```
# Idea del patrón, en una línea:  
# si ya existe instancia → devolvérsela; si no, crearla.
```

Críticas y advertencias

El Singleton es uno de los patrones más controvertidos del catálogo. Muchos desarrolladores experimentados lo consideran directamente un anti-patrón. Conviene conocer sus desventajas antes de elegirlo.

- **Estado global oculto:** un Singleton es estado global disfrazado de objeto. Esto dificulta razonar sobre el código, porque cualquier parte del sistema puede leerlo o modificarlo sin dejar rastro en las firmas de las funciones.
- **Complica el testing:** como la instancia persiste entre tests, los resultados pueden depender del orden de ejecución. Cada test debería resetear la instancia antes de correr.
- **Acoplamiento fuerte:** las clases que usan el Singleton dependen de su existencia y de su API. Cualquier cambio puede afectar a muchos lugares del código.
- **Problemas en concurrencia:** en aplicaciones con varios hilos hay que asegurar que la creación de la instancia sea thread-safe, lo que agrega complejidad.

Antes de usar Singleton — preguntate si realmente necesitás una única instancia, o si te alcanza con pasar el objeto como parámetro (lo que se conoce como inyección de dependencias). En la mayoría de los casos, pasarlo como parámetro es más limpio y más fácil de testear.

3. Factory: fábricas de objetos

Definición

El patrón Factory (también llamado Factory Method en su versión más estructurada) provee una interfaz para crear objetos sin que el código que los usa tenga que conocer la clase concreta que se está instanciando.

En vez de instanciar una clase concreta directamente, el código cliente le pide a una «fábrica» que le entregue el objeto. La fábrica decide qué clase usar en base a algún criterio (un parámetro, una configuración, el contexto de ejecución). El código cliente solo conoce la interfaz común que todas las variantes comparten.

¿Qué problema resuelve?

Imaginemos que estamos construyendo un sistema que procesa archivos de distintos formatos: CSV, JSON, Parquet, Excel. Sin Factory, en cada lugar donde haga falta abrir un archivo aparecería una larga cadena de condicionales decidiendo qué clase usar según el formato. Si más adelante quisiéramos sumar soporte para XML, tendríamos que tocar todos esos lugares.

Con Factory, esa lógica de decisión queda en un único punto del sistema. El resto del código se limita a pedirle al Factory un lector y trabajar con la interfaz común. Si mañana agregamos un nuevo formato, solo modificamos la fábrica.

```
# El código cliente nunca dice:  
lector = LectorCSV("ventas.csv")  
  
# Sino:  
lector = crear_lector("csv")
```

Ventajas del patrón Factory

- **Bajo acoplamiento:** el código cliente no depende de las clases concretas, solo de la interfaz común. Cambiar implementaciones internas no rompe a quien las usa.
- **Cumple el principio Open/Closed:** agregar un nuevo tipo solo requiere crear la clase nueva y registrarla en la factory. El código que la consume no se modifica.
- **Centraliza la lógica de creación:** si la instanciación requiere validar parámetros, leer configuración o calcular valores, esa complejidad queda encapsulada en un único lugar.
- **Facilita el testing:** podemos preparar una factory de prueba que devuelva mocks u objetos simplificados en lugar de los reales.

Regla práctica — cuando en el código aparece una cadena larga de «if formato == ... elif formato == ...» que decide qué clase instanciar, es muy probable que convenga refactorizarla a un Factory.

Variantes habituales

Factory es un patrón flexible y suele aparecer con varias formas según el contexto. Las tres más comunes son: una función simple que devuelve la clase apropiada (la más liviana, ideal para pocos tipos), una clase Factory con métodos estáticos (más estructurada y permite agrupar varias fábricas), y la versión clásica del Gang of Four en la que la fábrica es ella misma una clase abstracta con subclases que producen distintos objetos.

En el cuaderno de Colab vas a ver implementadas las dos primeras variantes, que son las más usadas en proyectos reales con Python.

4. Repository: abstracción del acceso a datos

Definición

El patrón Repository introduce una capa intermedia entre la lógica de negocio y la fuente real de los datos (una base de datos relacional, una API externa, un archivo CSV, un data lake). El Repository expone métodos con nombres del dominio del problema, como «obtener cliente por id», «guardar producto» o «listar pedidos activos», ocultando completamente cómo se almacenan o consultan esos datos por debajo.

Por qué nos importa especialmente — Repository no forma parte del catálogo clásico del Gang of Four, pero es probablemente uno de los patrones más usados en arquitecturas modernas, especialmente en Domain-Driven Design y Clean Architecture. En proyectos de ciencia de datos resulta clave porque permite cambiar la fuente de datos (por ejemplo, pasar de un CSV de prueba a una base PostgreSQL en producción) sin tocar la lógica que los consume.

¿Qué problema resuelve?

Sin Repository, la lógica de negocio termina mezclada con detalles de implementación de la base de datos. Una función que debería ocuparse de calcular el total de ventas de un cliente termina escribiendo consultas SQL, abriendo conexiones y manejando cursores. Esto trae varios problemas:

- Si cambia la base de datos (por ejemplo, de SQLite a PostgreSQL), hay que reescribir muchos lugares.
- Para testear la lógica de negocio se necesita una base real disponible, lo que vuelve los tests lentos y frágiles.
- Las mismas consultas aparecen repetidas en distintas funciones, con pequeñas variaciones que son difíciles de mantener consistentes.
- Una función que debería tener una única responsabilidad termina ocupándose también de la persistencia.

Cómo se estructura

La implementación típica del patrón consta de tres piezas que se relacionan entre sí:

1. **Las entidades del dominio:** clases que representan los conceptos del problema, como Cliente, Producto o Pedido. Son objetos de datos sin lógica de persistencia.
2. **La interfaz del Repository:** una clase abstracta que define qué operaciones debe ofrecer cualquier repositorio para esa entidad: obtener por id, guardar, listar, eliminar. Es el contrato.
3. **Una o más implementaciones concretas:** clases que respetan ese contrato pero que usan distintos mecanismos de almacenamiento. Por ejemplo: una en memoria (ideal para tests), una contra SQLite, una contra una API externa.

La lógica de negocio recibe el repositorio como parámetro (inyección de dependencias), sin enterarse de cuál implementación concreta se está usando. Esto permite cambiar la fuente de datos sin tocar la lógica que la consume.

```
# La lógica de negocio nunca sabe de SQL ni de archivos:
servicio = ServicioClientes(repo)
servicio.dar_de_baja(42)
```

Repository en proyectos de ciencia de datos

En un proyecto de ciencia de datos los datos suelen venir de fuentes muy distintas: un data warehouse corporativo, archivos parquet alojados en S3, una API externa, o simplemente un CSV de prueba que generamos en local. El patrón Repository permite que el código de exploración, entrenamiento o evaluación no dependa de la fuente concreta. Podemos tener un repositorio que lea desde un CSV para desarrollar rápido, otro que lea desde S3 para producción, y un tercero en memoria para los tests, todos respetando la misma interfaz.

El código de entrenamiento simplemente recibe un repositorio y le pide los features y las labels. Cambiar de fuente es cuestión de inyectar otro repositorio: el resto del código permanece igual.

Ventajas

- **Testabilidad:** los tests usan un repositorio en memoria, sin necesidad de levantar una base de datos. Son rápidos y deterministas.
- **Intercambiabilidad de fuentes:** cambiar de motor de persistencia o sumar nuevas fuentes no toca la lógica de negocio.
- **Centralización de consultas:** todas las consultas relacionadas con una entidad quedan en un único lugar, fáciles de optimizar y auditar.
- **Separación de responsabilidades:** el código de dominio expresa reglas del negocio, no detalles técnicos de almacenamiento.

5. Comparación: cuándo usar cada uno

Los tres patrones que estudiamos resuelven problemas distintos y se complementan muy bien entre sí. Vale la pena tener clara cuál es la pregunta a la que responde cada uno.

Singleton	Factory	Repository
Pregunta	Pregunta	Pregunta
¿Cómo garantizo que haya una sola instancia?	¿Cómo creo objetos sin atarme a una clase concreta?	¿Cómo aislo la lógica de negocio del origen de los datos?
Caso típico	Caso típico	Caso típico
Configuración global, logger, conexión a base de datos.	Lectores de archivos, notificadores, parsers según formato.	Acceso a base de datos, APIs externas, fuentes de datasets.
Riesgo principal	Riesgo principal	Riesgo principal
Estado global oculto que complica el testing.	Sobre-ingeniería si en realidad hay una sola clase.	Algo de boilerplate inicial, vale la pena en proyectos no triviales.

Los tres juntos en un mismo proyecto

Estos patrones se combinan muy bien. Un escenario habitual es el siguiente: tenemos una clase Configuración implementada como Singleton, que centraliza qué tipo de almacenamiento usar en cada entorno. Esa configuración la consulta una función Factory que decide qué Repository concreto entregar. Y finalmente, la lógica de negocio recibe el Repository por inyección de dependencias y trabaja con él sin saber nada del resto.

De esta forma, la lógica de negocio no sabe nada de configuración, ni del motor de base de datos, ni de cómo se decidió la implementación. Es desacoplamiento real: cada pieza tiene una responsabilidad única y cualquiera de ellas puede cambiarse sin tocar las demás.

Lo importante — los patrones por sí solos no son la meta. La meta es escribir código mantenible y fácil de testear. Los patrones son medios para lograrla.

Pregunta de reflexión — al terminar el ejercicio integrador, contestá: si mañana te piden cambiar de archivo JSON a una base PostgreSQL, ¿cuántas líneas de la lógica de negocio tenés que modificar? La respuesta ideal es cero.

6. Material complementario

Lecturas online

- Refactoring Guru, Singleton: <https://refactoring.guru/es/design-patterns/singleton>
- Refactoring Guru, Factory Method: <https://refactoring.guru/es/design-patterns/factory-method>
- Refactoring Guru, índice general: <https://refactoring.guru/es/design-patterns>
- Python Patterns Guide, Singleton (con análisis crítico): <https://python-patterns.guide/gang-of-four/singleton/>
- Martin Fowler, Repository Pattern: <https://martinfowler.com/eaCatalog/repository.html>
- Cosmic Python, capítulo sobre Repository (libro online gratuito): https://www.cosmicpython.com/book/chapter_02_repository.html
- Principios SOLID en español: <https://profile.es/blog/principios-solid-desarrollo-software-calidad/>

Libros recomendados

- Design Patterns: Elements of Reusable Object-Oriented Software, de Gamma, Helm, Johnson y Vlissides. Es el famoso libro del «Gang of Four» (1994), referencia obligada del tema.
- Patterns of Enterprise Application Architecture, de Martin Fowler. En este libro aparece formalizado el patrón Repository.
- Architecture Patterns with Python, de Percival y Gregory. Disponible gratis en cosmicpython.com. Muy práctico y con ejemplos modernos.

Conceptos relacionados para investigar

- **Inyección de dependencias (DI):** técnica que se usa junto al Repository. En vez de que una clase cree sus dependencias, las recibe por parámetro.
- **Unit of Work:** patrón complementario al Repository, especialmente útil para manejar transacciones cuando hay varios repositorios involucrados.
- **Principios SOLID:** los cinco principios fundamentales del diseño orientado a objetos. El patrón Repository encarna especialmente el principio de Inversión de Dependencias (la D de SOLID).
- **Anti-patrones:** malas prácticas comunes como God Object, Spaghetti Code o Magic Numbers. Reconocerlos ayuda a detectar código que necesita refactorización.

Programación Avanzada — Licenciatura en Ciencia de Datos, UNAB. Material de clase preparado por Franco Navarro y Agustin Almirón.